

Kaspar-HFT

Open Source

A CME futures trading system and order book simulator.

VENUE	MARKET DATA	ORDER ENTRY	BOOK	LICENCE
CME Globex	MDP3 SBE	iLink 3	MBP & MBO L3	MIT, open source

From production to simulation. It is a production trading system — multicast market data in, full order books reconstructed order by order, an execution algorithm on top, an order-management session out to the exchange. It is also an order book simulator: the same books rebuilt from recorded packet captures.

The same strategy code, the same execution algorithm and the same book run in PCAP replay, in live paper trading, and against the live exchange. Moving between them is a configuration change, so a backtest exercises the code path that will trade.

COMPONENTS

SIMULATION

Three modes in one binary: **PCAP replay**, live multicast paper trading and live iLink execution. Replay is deterministic: a session replayed twice gives identical fills.

CONCURRENCY

A **C++20 actor framework** with a formal concurrency model. The same code runs with actors on their own threads or grouped onto one thread, so the hot path can be spread out or kept on a single thread without code changes.

STRATEGY AUTHORING IN C++ OR RUST

C++ in-process actors for the lowest latency, or Rust through an in-process FFI. Either way the strategy runs in the same process as the book.

EXECUTION

Shadow-POV, a passive/aggressive percentage-of-volume algorithm that places orders by following other participants' orders in the market.

MARKET DATA

Full **MDP3** SBE decoder: incremental book updates, trades, order-by-order MBO, instrument definitions, snapshot recovery. Sequence gaps are detected and repaired automatically.

ORDER BOOK

Order-by-order reconstruction, holding every resting order individually. Queue position, and therefore fill inference, is exact.

ORDER ENTRY

iLink 3 session handler: SBE encoding, HMAC authentication, sequence management, negotiation and establishment, primary/secondary failover.

PERFORMANCE

AMD EPYC 9374F · RHEL 9 · g++ 15 · -O3 -march=native

Measured messaging costs of the actor framework. `fast_send` runs the receiver's handler inline on the caller's thread and returns the reply as a value.

PATH	P50	NOTES
<code>fast_send</code> — inline, same thread	~10 ns	≈ one polymorphic virtual call
Queued send, one Group thread	~90 ns	queue push/pop + dispatch
Queued send, separate threads	~3,370 ns	two cross-core mailbox wakeups
Pooled round trip (2 messages)	~92 ns	vs ~128 ns with plain <code>new</code>
Allocator tail (max observed)	~33 μs	vs ~5.5 ms without the pool

MEASURED Round trip through the framework — send, handle, reply — one message in flight, from the microbenchmarks in `actors/cpp/perf`.